

 Чему научиться в этом году?

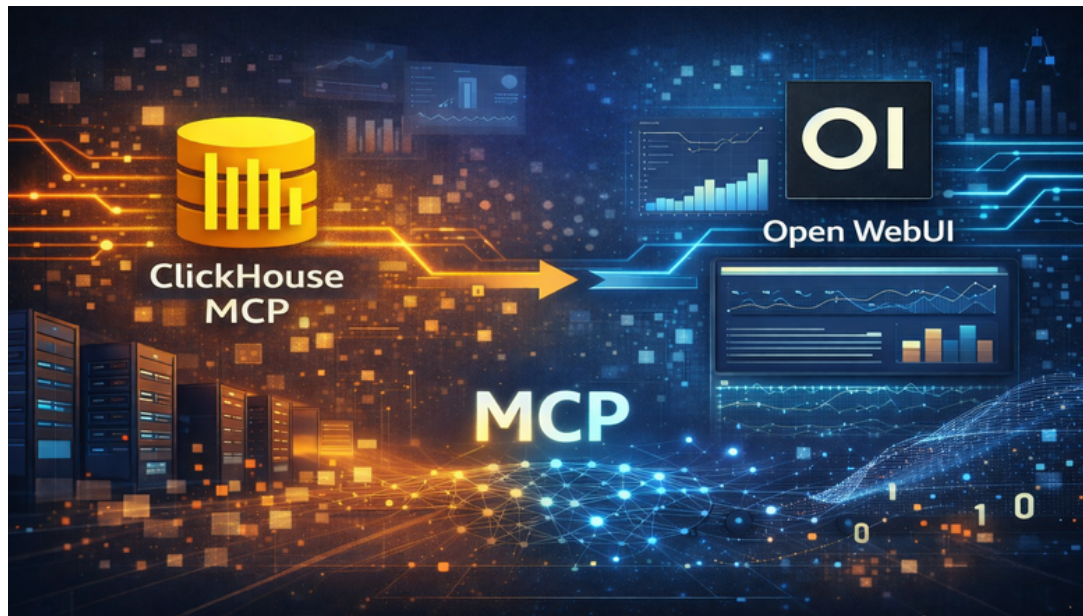
minitower
26 мар в 02:10

Реализация MCP в Open WebUI. Часть 2 — Агентское поведение

 Средний  19 мин  4.7K

Python*, Базы данных*, Машинное обучение*

Кейс



Обложка

Содержание

1. Введение
2. Наивная реализация агента: дам инструмент и все взлетит
 - 2.1. Реализация на существующем MCP
 - 2.2. Реализация собственных инструментов
3. Проблемы наивного подхода к реализации агента
4. Вторая реализация агента: планируй и делай
 - 4.1. Полнотекстовый поиск как фильтрация больших данных
 - 4.2. Планировщик как отдельная часть агента: метод ReWOO
5. Заключение

Для создания обложки данной статьи использовался ИИ. В ходе написания ИИ не использовался, вся статья является описанием собственного опыта, основанного на различного рода ошибках и проблемах.

Введение

Всем привет!

Это вторая часть нашей реализации MCP на Open WebUI, которая строится для целей портфельной аналитики. В прошлой части мы разобрали интеграционную часть нашего решения: как мы пришли к Open WebUI, как использовали статусы в запросах пользователя, как отображаем результаты (чтобы не словить ошибки о слишком больших чанках), как строим графики в интерфейсе и как работаем с запросами пользователей. Данная часть будет посвящена реализации самого агента: его общению с инструментами, вызову этих самых инструментов и планированию шагов по их вызовам.

В данной статье будет рассмотрен наш путь по реализации данного агента: от наивного агента, которому просто дали пул инструментов и отправили в релиз до разделения агента на планировщик и исполнителя (спойлер: вторая версия стала куда лучше справляться с различными задачами). Отдельно будет затронута возможность различных моделей использовать chain-of-thoughts.

Последняя реализация данного агента позволяет выполнять несколько последовательных запросов и потом отдавать результат пользователю, как это было представлено в первой статье. Таблицы соединяются в одну, а сам агент решает, какие таблицы нужно показывать пользователю.

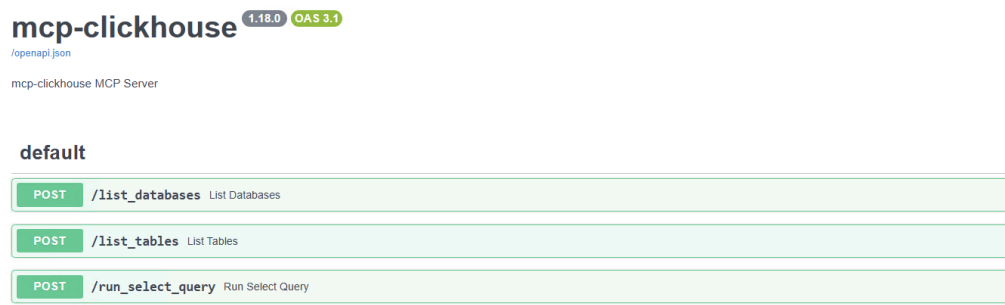
Наивная реализация агента: дам инструмент и все взлетит

Реализация на существующем MCP

Казалось бы, что на этапе победы проблемы с большими данными все дальнейшее должно пройти просто: нужно дать агенту список инструментов, научить его с ними общаться и наблюдать, как он их корректно вызывает и отдает идеальные результаты.

Изначально в ClickHouse MCP предусмотрен только 3 инструмента:

- list_databases - список баз данных, которые доступны для агента;
- list_tables - список таблиц, которые доступны для агента;
- run_select_queries - выполнить SQL запрос в БД. Интерфейс MCP ClickHouse.png



Интерфейс MCP ClickHouse

Для каких-либо простых запросов (например, выбрать уникальные портфели из таблицы) этих инструментов с лихвой хватало для нужд MCP. Однако когда запросы становились сложнее, то стало понятно, что она не понимает структуру данных (возможно, дело в том, что модели не очень большие и не имеют сильных аналитических возможностей).

Первым нашим решением было написать ему промпт, в котором привели список запросов и как их нужно писать. Промпт выглядел примерно следующим образом:

```
{examples:
  [{
```

```

prompt: "Найди доходность портфеля {portfolio_name} за период с {start_date} по {end_date}"
query: WITH log_coef as (SELECT Portfolio, Dt,
    sum(log(TWR_dod+1)) OVER (
        PARTITION BY InvestmentPortfolioID
        ORDER BY Dt ASC ROWS BETWEEN UNBOUNDED PRECEDING AND 0 FOLLOWING
    ) AS TWR_cumulative_coef
FROM Contribution.contribution_twr_1s_mcp
WHERE lowerUTF8(Portfolio) LIKE '%{portfolio_name}%'
    AND Dt::date >={start_date}
    AND Dt::date <= {end_date}
ORDER BY Dt DESC)
SELECT Portfolio, Dt, exp(TWR_cumulative_coef) - 1 AS TWR_cumulative
FROM log_coef;

,
"tool_response": {
    "status": "success",
    "table": "contribution_twr_1s_mcp",
    "columns": [
        {"name": "TWR_cumulative", "type": "Float64", "comment": "Доходность"}
    ],
    "count": 1,
    "rows": [
        {"TWR_cumulative": {query_result}}
    ]
},
"assistant_final": "Доходность для портфеля {portfolio_name} за период с {start_date} по {end_date}: {TWR_cumulative}"
],
-- Список других примеров для LLM --
],
"variables": {
    "portfolio_name": "Название портфеля для поиска (может быть частичным совпадением)",
    "start_date": "Начальная дата периода в формате YYYY-MM-DD",
    "end_date": "Конечная дата периода в формате YYYY-MM-DD",
    "portfolio_list": "Список портфелей для сравнения через запятую",
    "date": "Дата из запроса пользователя в формате YYYY-MM-DD",
    "query_result": "Результат выполнения запроса от MCP",
    "portfolio_type": "Тип портфеля"
},
"instructions": "Используй переменные {portfolio_name}, {start_date}, {end_date} в запросах",
"common_patterns": {
    "single_portfolio_filter": "Portfolio LIKE '%{portfolio_name}%',
    "date_range_filter": "Dt::date >= '{start_date}' AND Dt::date <= '{end_date}'",
    "multiple_portfolios": "Используй CROSS JOIN с таблицей шаблонов для сравнения"
}
}

```

[Объяснить с 🧠](#)

Были 2 попытки использовать данных промпт: подключить RAG систему (использовали ту, что заложена в Open WebUI) с обязательным добавлением строки с variables и instructions, а также в качестве полного промпта. Результаты были примерно одинаковыми: модель действительно начинала писать запросы лучше, но все еще не детерминировано: частые ошибки в синтаксисе и ошибки в понимании некоторых необходимых метрик говорили о том, что до продового решения еще было далеко.

Было принято решение, что 3 инструментов для работы с БД нам определенно недостаточно. Мы стали расширять набор инструментов для LLM и дописывать существующий MCP.

Реализация собственных инструментов

Для детерминированного результата данного агента мы решили проблему кардинально: дать ему уже готовые инструменты, которые просто ходили в БД и выполняли заранее готовые запросы. Так LLM не имела пространства для галлюцинаций, что позволяло получать детерминированные результаты при каждом новом проходе.

Инструменты я разбил по их логике: так появились инструменты:

- ClickHouseClientBase - инструмент с исходными инструментами ClickHouseMCP;
- ProfitTool - инструмент с запросами относительно доходностей определенных портфелей (доходности по датам, средней арифметической и средней геометрической доходностями);
- PortfolioDiscoveryTool - инструмент с информацией относительно портфелей: список уникальных портфелей, типов портфелей и прочими атрибутами портфелей (например, к каким стратегиям относятся);
- PortfolioCashflowTool - инструмент с притоками/оттоками по портфелям и СЧА по данным портфелям. По сути все они состоят из pydantic схемы, которая содержит необходимые для инструмента параметры и инструкции относительно того, как их нужно задавать:

```
class ClickHouseClientBaseParams(BaseModel):
    """Параметры для инструмента взаимодействия с ClickHouse"""
    operation: Literal['list_databases', 'list_tables', 'run_select_query'] = Field(
        description='Тип операции: list_databases, list_tables или run_select_query'
    )
    database: Optional[str] = Field(
        default=None,
        description='Имя базы данных (требуется для list_tables)'
    )
    query: Optional[str] = Field(
        default=None,
        description='SQL-запрос для выполнения (требуется для run_select_query)'
    )
    like: Optional[str] = Field(default=None, description='Фильтр LIKE для list_tables')
    not_like: Optional[str] = Field(default=None, description='Фильтр NOT LIKE для list_tables')

def clickhouse_client_base(params: ClickHouseClientBaseParams) -> str:
    """Инструмент для взаимодействия с ClickHouse: список баз данных, список таблиц и т.д.
    base_url = 'http://clickhouse-mcp.services.kfim.int'

    try:
        if params.operation == 'list_databases':
            response = requests.post(f'{base_url}/list_databases')
            response.raise_for_status()
            return json.dumps(response.json(), ensure_ascii=False)

        elif params.operation == 'list_tables':
            if not params.database:
                return json.dumps({'error': 'Parameter "database" is required for list_tables'})

            payload = {'database': params.database}
            if params.like:
                payload['like'] = params.like
            if params.not_like:
                payload['not_like'] = params.not_like

            response = requests.post(f'{base_url}/list_tables', json=payload)
            response.raise_for_status()
            return json.dumps(response.json(), ensure_ascii=False)
```

```

elif params.operation == 'run_select_query':
    if not params.query:
        return json.dumps({'error': 'Parameter "query" is required for run_select_query'})

    query = params.query
    if 'Contribution.contribution_twr_1s_mcp' not in query \
        and 'contribution_twr_1s_mcp' in query:
        query = query.replace('contribution_twr_1s_mcp', 'Contribution.contribution_twr_1s_mcp')
    payload = {'query': query.replace(' ', '').replace('\n', ' ').replace('\\', '\\')}
    response = requests.post(f'{base_url}/run_select_query', json=payload)
    return json.dumps(response.json(), ensure_ascii=False)

else:
    return json.dumps({'error': f'Unknown operation: {params.operation}'}, ensure_ascii=False)

except json.JSONDecodeError as e:
    return json.dumps({'error': f'Failed to parse response: {str(e)}'}, ensure_ascii=False)
except Exception as e:
    return json.dumps({'error': f'Error: {str(e)}'}, ensure_ascii=False)

```

[Объяснить с](#) 

Внутри каждого инструмента прописано некоторые правила как работать с параметрами инструмента и какие параметры нужны для каждой конкретной операции.

Т. к. конкретный инструмент невозможно без преобразований передать в модель была написана отдельная функция, которая преобразовывала выбранную модель в OpenAI схему, с которой уже и работает LLM-ка для вызова инструментов. Вот так выглядит преобразование:

```

def pydantic_to_openai_schema(pydantic_model: type[BaseModel], function_name: str, description: str) -> dict:
    """Convert a Pydantic model to OpenAI function calling schema"""
    schema = pydantic_model.model_json_schema()

    # Extract the parameters schema (properties and required)
    parameters_schema = {
        "type": "object",
        "properties": schema.get("properties", {}),
    }
    if "required" in schema and schema["required"]:
        parameters_schema["required"] = schema["required"]

    # OpenAI function schema format
    return {
        "type": "function",
        "function": {
            "name": function_name,
            "description": description,
            "parameters": parameters_schema
        }
    }

```

[Объяснить с](#) 

И данный подход начал работать: запросы обрабатывали штатно, функции выполнялись в соответствии с тем, что в них было заложено. После получения первых хороших результатов мы дали попользоваться модель пользователям...

Проблемы наивного подхода к реализации агента

Полетели первые запросы от пользователей. В большинстве случаев модель (на тот момент у нас была развернута gpt-oss-20B) справлялась хорошо. Однако были свои нюансы:

1. Пользователь неправильно написал имя портфеля. Так, например, можно было пропустить одну букву в имени портфеля и весь скрипт ломался, потому что указанного портфеля не было представлено. При этом LLM не могла указывать на ошибку в написании данного портфеля, потому что она не знает всего списка портфелей (у нее есть инструмент для разведки портфелей, но при опечатке он не работает корректно);
2. Пользователь задал "нетипичный запрос". Так например запрос: выведи таблицу с колонками (NAV, Inflow, Outflow) по портфелю xx за период не мог отработать корректно, потому что не было переменной state, которая бы хранила предыдущие выполненные шаги. Агент просто выполнял запросы по очереди без их сохранения и выдавал последний вариант. Данные проблемы вскрыли проблему наивной реализации агента: она не позволяет исполнять несколько последовательных запросов и анализировать их результаты.

Вернее будет следующее утверждение: мой алгоритм реализации визуализации не позволял извлечь результаты из всех запросов. Подробнее о реализации визуализации можно прочитать в [первой части](#)

Таким образом мы пришли к поиску современных практик по реализации агентов. Проблемы в целом было 2:

1. Портфелей достаточно много для того, чтобы была возможность LLM просмотреть их все. Контекст просто забивался и LLM не могла его корректно обработать. В следствие этого необходимо было их "отобрать" на этапе базы данных;
2. Необходимо, чтобы модель имела возможность спланировать свои действия до прямой их реализации. Таким образом модель могла бы выполнять несколько действий подряд не теряя контекста при этом. Для решения этих проблем были предприняты 2 новых архитектурных решения.

Вторая реализация агента: планируй и делай

Полнотекстовый поиск как фильтрация больших данных

Первой задачей стала возможность фильтрации количества портфелей (это задача нашего кейса, но кажется, что в любой реализации нужно будет фильтровать сущности для того, чтобы не забивать контекст и оставлять возможность для поиска). Это показалась как классическая задача *полнотекстового поиска*.

Так как мы используем ClickHouse как основную аналитическую базу данных мы решили сделать полнотекстовый поиск именно для этой базы данных. Т. к. реализация данного поиска [уже реализована](#) мы использовали именно эту реализацию.

После создания необходимого для данной реализации текстового индекса нужно было сделать запрос, который бы выводил наиболее похожие портфели. Для этого была написана следующая основная функция поиска:

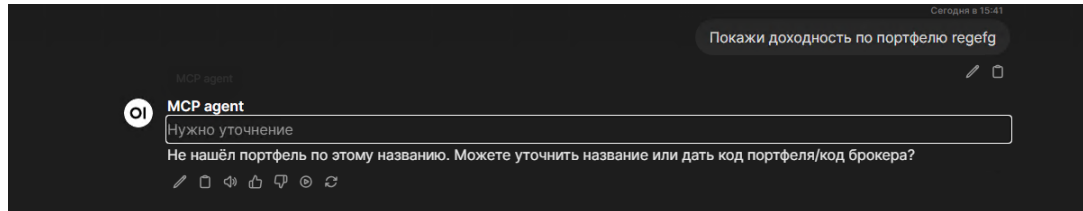
```
SELECT DISTINCT
  Portfolio,
  PortfolioCode, PortfolioClientCode,
  ngramDistanceUTF8(lowerUTF8(Portfolio), lowerUTF8('{portfolio}')) as distance
FROM Contribution.contribution_twr_1s_mcp
```

```
WHERE hasAnyTokens(Portfolio, lowerUTF8('{portfolio}'))  
ORDER BY distance
```

[Объяснить с](#) 

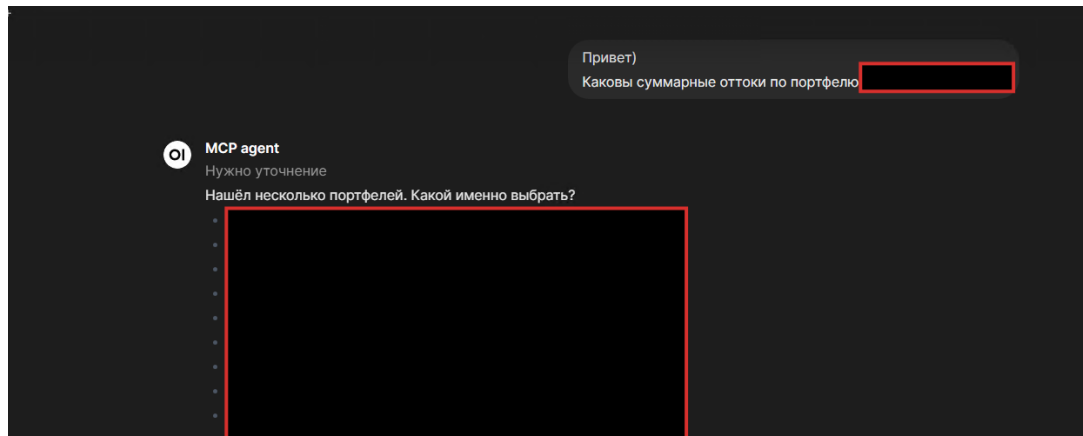
В целом алгоритм поиска портфеля работает следующим образом:

- Если портфелей в результате данного поиска не обнаружено, то пользователю отправляется clarification request, в котором сказано, что данного портфеля не найдено и нужно уточнить название портфеля:



Уточняющий вопрос в интерфейсе

- Если портфелей в результате данного поиска более чем 1, то пользователю отправляется clarification request, в котором предложено выбрать один из нескольких указанных портфелей:



Уточняющий вопрос на много портфелей

- Если портфель найден и он один, тогда запрос выполняется корректно и алгоритм проходит дальше.

Планировщик как отдельная часть агента: метод ReWOO

Второй важной архитектурным обновлением нашего агента стал метод ReWOO. Этот метод позволяет разделить этап планирования и исполнение этого плана. Для его реализации была написана логика планировщика и исполнителя:

1. Планировщиком является сама LLM. Она строит некоторый план, по которому в дальнейшем придется функция Python (исполнитель) и выполнит нужные инструменты;
2. Исполнитель не придумывает ничего нового. Он просто проходит по существующему плану и исполняет его;
3. Если планировщик попадает в ошибку (неправильно выбранные аргументы, неправильные результаты), то задача из исполнителя возвращается назад к планировщику и он анализирует ошибку и исправляет план (эта часть пока не реализована).

▸ [Вариант промпта](#)

Ты — планировщик. Верни ТОЛЬКО один JSON-объект (без markdown, без комментариев).

Цель: составить пошаговый план получения данных для ответа пользователю с помощью доступных инструментов.

Ограничения:

- Не вызывать инструменты (ты только планируешь).
- Используй только инструменты из `tools_registry`.
- Если пользователь не указал даты, используй диапазон 2025-01-01 ... 2025-12-31.
- Даты должны быть строго YYYY-MM-DD.
- Названия портфелей в планируемых параметрах — в нижнем регистре.

ВАЖНО: Различение типов портфелей и названий портфелей:

- Типы портфелей: “ДУ”, “ПИФ” или другие типы из БД (это категории портфелей).
- Названия портфелей: конкретные имена типа “пример”, “пример_2” и т.д.
- Если пользователь сказал “портфели типа ДУ” или “ДУ портфели” → это `portfolio_types: [“ДУ”]`.
- Если пользователь сказал “портфель ДУ” без слова “тип” → это может быть название портфеля “ДУ” (нужен `resolve_portfolio`).
- Если пользователь сказал “портфели ДУ” → скорее всего это тип, но лучше сначала проверить через `get_portfolios_type`.
- Если пользователь явно указал “тип портфеля” или “портфели типа X” → это `portfolio_types`.

Обязательное поведение:

- Если пользователь дал только “человеческое” имя портфеля (не коды, не тип), сначала запланируй шаг `resolve_portfolio` через `portfolios_discovery_tool(get_portfolios({portfolio}))`, где `{portfolio}` - это название от пользователя.
- Если пользователь указал что-то похожее на тип портфеля (ДУ, ПИФ, или “тип X”), но не уверен — запланируй шаг `validate_types` через `portfolios_discovery_tool(get_portfolios_type)` для проверки.
- Если после `resolve` возможна неоднозначность (несколько портфелей), планируй `clarification_question` и остановку (не выдумывай портфель).
- Если пользователь дал `portfolio_codes` или `portfolio_client_codes`, `resolve_portfolio` можно пропустить.
- Если пользователь явно указал `portfolio_types` (например “тип ДУ”), используй `portfolios_discovery_tool(get_portfolios_by_type)` или `profit_tool` с `portfolio_types`.
- В переменной `show_data` необходимо указывать те данные, которые запросил пользователь. Переменные, которые относятся к исследованию (например, `portfolio_info`), должны иметь `show_data=False`

Формат JSON: json, который приложен ниже

Для сохранения structured output просим модель отдавать информацию в виде JSON. Мной был спроектирован его следующий вид (эта часть идет в промпт):

```
{
  "needs_tools": true|false,
  "intent": "profitability|cashflows|discovery|other",
  "entities": {
    "metric": "profit|mean_profit|geometric_mean_profit|nav|inflow|outflow|portfolios_...",
    "portfolio_hint": "строка",
    "portfolio_names": "["..." | []",
    "portfolio_types": "["..." | []",
    "portfolio_codes": "["..." | []",
    "portfolio_client_codes": "["..." | []",
    "start_date": "YYYY-MM-DD",
    "end_date": "YYYY-MM-DD",
    "by_date": true|false
  },
  "steps": [
    {
      "id": "string",
      "tool": "tool_name",
      "args": { "..." : "..." },
      "save_as": "state_key",
      "show_data": True|False }
  ], "clarification_question": "string|null" }
```

[Объяснить с](#) 

В результате структурирования плана и его форматирования LLM выдает план следующего формата:

```
{
  "needs_tools": true,
  "intent": "profitability",
  "entities": {
    "metric": "profit",
    "portfolio_hint": "regefg",
    "portfolio_names": [],
    "portfolio_types": [],
    "portfolio_codes": [],
    "portfolio_client_codes": [],
    "start_date": "2025-01-01",
    "end_date": "2025-12-31",
    "by_date": false
  },
  "steps": [
    {
      "id": "resolve_portfolio",
      "tool": "portfolios_discovery_tool",
      "args": {
        "operation": "get_portfolios",
        "portfolio": "портфель_от_пользователя"
      },
      "save_as": "portfolio_info",
      "show_data": false
    },
  ],
  {
```

```
"id": "get_profitability",
"tool": "profit_tool",
"args": {
  "operation": "get_profit_of_portfolio",
  "portfolios": ["${resolve_portfolios}"],
  "start_date": "2025-01-01",
  "end_date": "2025-12-31"
},
"save_as": "profitability_data",
"show_data": true
}
],
"clarification_question": null
}
```

[Объяснить с](#) 

В дальнейшем его подхватывает планировщик, исполняет, и записывает состояния в state. После удачного запрос state выглядит примерно следующим образом:

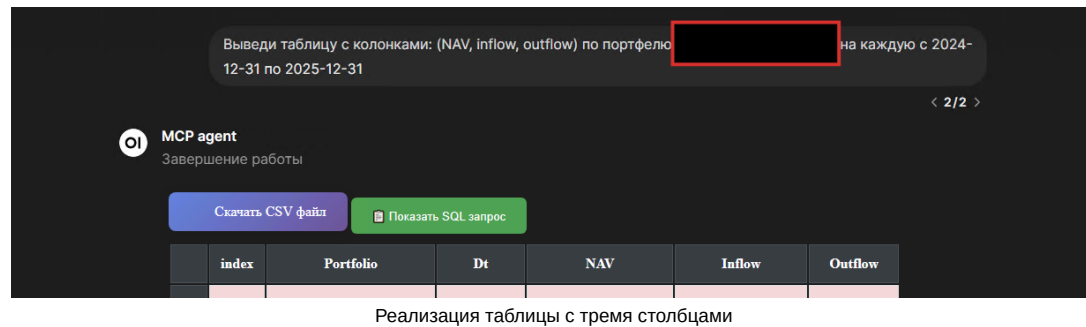
```
{'status': 'ok',
'state': {
  'step_results': [
    {
      'id': 'resolve_portfolio',
      'tool': 'portfolios_discovery_tool',
      'args': {
        'operation': 'get_portfolios',
        'portfolio': 'волга-капитал'
      },
    },
    'result':
    {'columns': ["Portfolio", "PortfolioCode", "PortfolioClientCode"], "rows": [['
'show_data': False
  ],
  {'id': 'get_profitability_chart',
   'tool': 'profit_tool',
   'args': {
     'operation': 'get_profit_of_portfolio',
     'portfolios': ['имя'],
     'start_date': '2025-01-01',
     'end_date': '2025-12-31',
     'by_date': True
   },
   'result': данные,
   'show_data': True
  }
}
```

[Объяснить с](#) 

Особенным показателем для целей дальнейшей сборки данных и показа их пользователям является показатель `show_data`. Именно он позволяет собрать полную необходимую табличку следующим алгоритмом:

1. Собираются все данные, которые были переданы в `state`;
2. Данные джойнятся по некоторой логике столбцов, которые были переданы в результате state;
3. Пользователю показываются все данные, которые LLM отметила как `show_data`.

В результате получается выдавать таблички наподобие той, что представлена ниже:



Выглядит это примерно следующим образом:

1. Модель создает план, в котором есть шаг разведки портфеля, а также 3 запроса к NAV, Inflow, Outflow;
2. Планировщик проходит по 3 запросам и собирает данные с флагом `show_data` ;
3. Финальный пайплайн собирает все необходимые данные и собирает их в финальную таблицу исходя из основных столбцов данной таблицы.

Заключение

Развитие агента в этом проекте наглядно показывает важную вещь: сами по себе инструменты не делают систему «умной». Наивный подход — дать LLM доступ к базе и ждать корректных ответов — работает только на простых сценариях. Как только появляются реальные пользовательские запросы, всплывают фундаментальные ограничения: отсутствие планирования, слабая работа с неопределенностью (опечатки, неоднозначности) и неспособность управлять состоянием.

Переход ко второй архитектуре стал качественным скачком. Добавление полнотекстового поиска решило проблему масштабируемости и «зашумленного» контекста, а разделение на планировщик и исполнитель (через ReWOO) дало агенту то, чего ему критически не хватало — структуру мышления. Теперь система не просто реагирует, а последовательно решает задачу: сначала определяет, что нужно сделать, затем делает это шаг за шагом, сохраняя промежуточные результаты.

В конечном итоге этот подход дал следующие положительные эффекты:

- Появилась возможность выполнения нескольких последовательных запросов с дальнейшим выводом результатов их выполнения;
- Увеличилась частота правильных вызовов инструментов для LLM и ответов агента как такового;
- Модель стала мыслить в контексте инструментов на несколько шагов вперед, что определило ее взаимодействие с ними всеми.

Также у нас есть план действий по улучшению текущего решения. То, что было замечено в ходе тестирования:

1. Несмотря на то, что LLM стала лучше ориентироваться в инструментах и вызывать их ей не хватает глобального перепланировщика (при возникновении ошибки executor-а следует отдать модели результаты после executor-а и попросить прописать шаги заново);
2. Добавление новых инструментов будет неизменно вредить качеству модели. Необходимо использовать zero-shot classifier модель для того, чтобы была возможность строить некоторые

маршруты для определенного скопа инструментов;

3. Добавление валидации до исполнения агрегатов самой моделью. Реализация планируется чистыми функциями Python, которые будут отсеивать вредные паттерны и отправлять в перепланировщик.

В третьей части хочется затронуть использование различных LLM в нашей задаче, которые нам уже довелось попробовать и в каких задачах они стали лучше или хуже и почему. Нам удалось уже попробовать:

- GLM-4.7;
- Qwen-3;
- Qwen-3.5;
- gpt-oss;
- и некоторые другие

Я надеюсь прочтение этой статьи дало вам немного больше информации о разработке агентских сетей и вы нашли для себя что-то полезное. Спасибо всем, кто дочитал мою статью и до скорых встреч!

Только зарегистрированные пользователи могут участвовать в опросе. [Войдите](#), пожалуйста.

Какая структура построения агента кажется вам наиболее перспективной?

0%	Да и наивный неплох, сделал за 5 минут и доволен	0
0%	Базовая рефлексия is all you need	0
100%	Планировщик и executor — это основа любой агентской системы	1
0%	Я строю агентов под высокими нагрузками и только на ациклических графах	0

Проголосовал 1 пользователь. Воздержался 1 пользователь.

Теги: [llm-агент](#), [mcp](#), [clickhouse](#), [open-webui](#), [аналитика](#), [автоматизация](#)

Хабы: [Python](#), [Базы данных](#), [Машинное обучение](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

3

Охват за 30 дней

Карма

Влад Алферов @minitower

Аналитик данных и ML инженер

Подписаться



GitHub

Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ

**runetfreedom**

9 часов назад

Из-за критической уязвимости VLESS клиентов скоро все ваши VPN будут заблокированы

Простой



5 мин



135K

+319

269

410

**zarazaexe**

10 часов назад

BAREBONE2022: чтобы заблокировать этот протокол придется запретить MAX и Yandex

Простой



4 мин



40K

+94

147

62

**Firemoon**

8 часов назад

Вспоминаем молодость: как делались скриншоты на старых кнопочных Nokia

9 мин



6.5K

+33

6

2

**dalerank**

22 часа назад

Лишние вычисления

Простой



12 мин



12K

+31

46

18

**TrexSelectel**

4 часа назад

Беспроводная связь в ядерном пекле: как японцы заставили Wi-Fi выживать там, где умирает вся электроника

4 мин



4.7K

+28

6

0



k0mar0v

7 часов назад

Commodore OS Vision 3.0 — операционка для фанатов ретрогейминга



6 мин



4.8K



+22



7



0



melnik909

7 часов назад

9 CSS-лайфхаков для улучшения пользовательского опыта



Средний



6 мин



5K

Обзор



+20



14



1



AmirUr

7 часов назад

Управление SSH-доступом в 2026 году: от зоопарка с jump-host и Ansible к единой точке входа в инфраструктуру с Warpgate



Средний



6 мин



6.8K

Кейс



+17



33



4



maxim_tsar

9 часов назад

МАИ: как здесь учились конструкторы первых космических ракет



11 мин



4.4K



+16



5



0



Sec-Explorer

6 часов назад

«Олег, разверни тестовую базу»: как таска на 5 минут сорвала финтех-релиз и поссорила три отдела



6 мин



5.6K

Кейс



+15



12



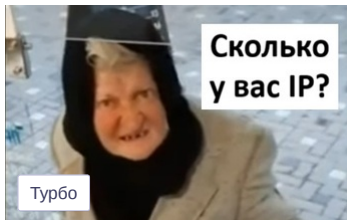
7

Результаты исследования: как айтишники относятся к ИИ

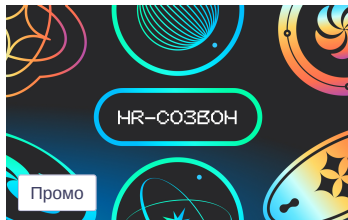
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Сотням ВМ — десяток IP. И всё работает



Как сделать так, чтобы сотрудники говорили о компании хорошее



Отношение айтишников к ИИ: результаты исследования

ВАКАНСИИ

Web-аналитик / Веб-аналитик
от 120 000 до 250 000 ₽ · ElsiKora · Можно удаленно

Automation & AI Integration Engineer
от 100 000 до 120 000 ₽ · ДЕТКИ · Новомосковск · Можно удаленно

Системный аналитик (Мои Сервисы)
от 300 000 ₽ · Сбер · Москва

Системный аналитик
от 130 000 ₽ · Synapse AI · Москва · Можно удаленно

ML-инженер
от 4 500 до 7 500 \$ · DataLouna · Можно удаленно

[Больше вакансий на Хабр Карьере](#)

ЧИТАЮТ СЕЙЧАС

Из-за критической уязвимости VLESS клиентов скоро все ваши VPN будут заблокированы

135K 410

Отчёт Cloudwards: Россия на предпоследнем месте по уровню свободы интернета в 2026 году

123K 202

Вышел клиент Telegram на Android v12.6.4 (6666), в апдейте исправили проблему с работой MTProto

62K 59

BARBONE2022: чтобы заблокировать этот протокол придется запретить MAX и Yandex

40K 62

Инцидент в сети «Ростелеком» с недоступностью сервисов произошёл из-за мощной DDoS-атаки

44K 36

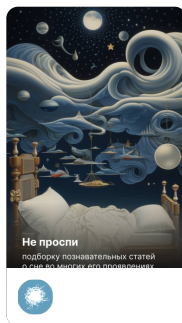
Результаты исследования: как айтишники относятся к ИИ

Турбо

ИСТОРИИ



Лучшее из блогов компаний
за неделю



Самое время поспать



Просто взять и пошутить



Со статьями о Heavy Digital —
на завод



Пресс, который мы
заслужили



БЛИЖАЙШИЕ СОБЫТИЯ



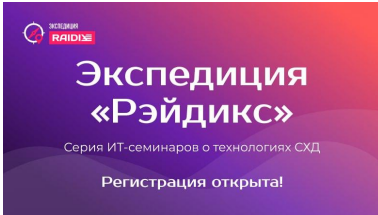
1 марта – 9 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

Больше событий в календаре



2 марта – 8 июня

Экспедиция «Рэйдикс»

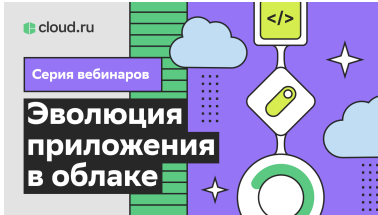
Серия ИТ-семинаров о технологиях СХД

Регистрация открыта!

Екатеринбург • Новосибирск • Краснодар • Минск • Калининград • Ростов-на-Дону

Разработка Другое

Больше событий в календаре



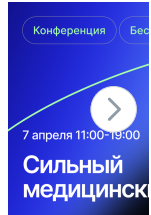
18 марта – 27 апреля

Серия вебинаров «Эволюция приложения в облаке»

Онлайн

Разработка

Больше событий в календаре



6 апреля

Онлайн-«Сильный бренд»

Онлайн

Менеджмент

Другое

Больше событий в календаре

Ваш аккаунт	Разделы	Информация	Услуги
Войти	Статьи	Устройство сайта	Корпоративный блог
Регистрация	Новости	Для авторов	Медийная реклама
	Хабы	Для компаний	Нативные проекты
	Компании	Документы	Образовательные программы
	Авторы	Соглашение	Стартапам
	Песочница	Конфиденциальность	



Настройка языка

Техническая поддержка